division with the remainder property, define the integers r0 , r1 , . . . , rλ+1 , and q1 , . . . , qλ , where λ ≥ 0, as follows:

```
a = r0 .
b = r1,
r0 = r1 q1 + r2    (0 < r2 < r1 ),
  .
  .
  .
r1−1 = r1 q1 + r1+1   (0 < r1+1 < r1 ),
  .
  .
  .
rλ−2 = rλ−1 qλ−1 + rλ (0 < rλ < rλ−1 ),
rλ−1 = rλqλ.          (rλ+1 = 0).
```

From the given definitions λ = 0 if b = 0, and λ > 0 for all other cases. Then we have rλ =√gcd(a, b). Moreover, if b > 0, then λ ≤ log b/ log φ + 1, where φ := (1 + 5)/2 ≈ 1.62.

Algorithm: If we could furnish input as a and b, where a and b are integers such that a ≥ b ≥ 0, we can compute d = gcd(a, b) as exemplified below:

```
r ← a, r ← b
while r = 0 do
    r ← r mod r
    (r, r ) ← (r , r )
d← r
output d
```

Designing algorithms for specific problems

We have the Legendre differential equation as:

$$\frac{d}{dx}\left[(1-x^2)\frac{d}{dx}P_n(x)\right] + n(n+1)P_n(x) = 0$$

Also, please note that the Legendre equation may be represented by:

$$ax^2 + by^2 + cz^2 = 0$$

And we also have the Sturm-Liouville equation that is a real second-order linear equation of the form:

$$-\frac{d}{dx}\left[P(x)\frac{dy}{dx}\right] + q(x)y = \lambda w(x)y$$

Here, y is a function of the free variable x, and p(x), q(x) and w(x) are the functions of x. We will be handling the Sturm-Liouville problem that intends to find the value of λ . Please refer to ***mathworld.wolfram.com/Sturm-LiouvilleEquation.html*** for more information. Now our concern is to build an algorithm. Let's do this in C.

Problem: To solve the Legendre equation with the simplest algorithm for the Sturm-Liouville equation,

```
/* Solving the Legendre equation with the simplest algorithm for the Sturm-
Liouville equation
Code written for A Voyage to Kernel - 8*/

#include <stdio.h>
#include <math.h>
#define NOMAX 1000

main()
{
int i,no,inc;
double dl=1e-5;
double u[NOMAX];
double z,r,s,uu,t,f0,f1;
void salgo();

/* Initialization of our problem */

no = NOMAX;
z = 3.0/(no-1);
s = 1.5;
r = 0.5;
t = 0.5;
inc = 0;
u[0] = -1;
u[1] = -1+z;
uu = r;
salgo (no,z,uu,u);
f0 = u[no-1]-1;
while (fabs(t) > dl)
 {
 uu = (r+s)/2;
 salgo (no,z,uu,u);
 f1 = u[no-1]-1;
 if ((f0*f1) < 0)
  {
  s = uu;
  t = s-r;
  }
 else
  {
  r = uu;
  t = s-r;
  f0 = f1;
  }
 inc = inc+1;
 }
printf("%4d %16.8lf %16.8lf %16.8lf\n", inc,uu,t,f1);
```